Compared to the last notebook, now we're going to make a custom Aerosol Layer to represent a hazy model.

Photochemical haze formation is complex and there are fewer (if any) codes that parametrize haze formation the way Virga does cloud formation.

In this tutorial you will learn:

1. How to compute the optics of an aerosol for your custom radius grid 2. How to inject this aerosol as a custom layer into an atmospheric model for PICASO

You need to have downloaded PICASO, Virga, a stellar model file from the PHONEIX grid, and the particle size distribution file to run this tutorial.

First, let's again import all the necessary packages:

Code cell 3

```
#standards
from pathlib import Path
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import astropy.units as u
from astropy.io import fits

workshop_dir = Path.cwd()
if not (workshop_dir / 'picaso_reference').exists():
    workshop_dir = workshop_dir.parent
reference_dir = workshop_dir / 'picaso_reference'
os.environ['picaso_refdata'] = str(reference_dir)
os.environ['PYSYN_CDBS'] = str(workshop_dir / 'data' / 'stellar')

# Download gas opacities if they are not already in the workshop folder.
opacity_file = reference_dir / 'opacities' / 'opacities_0.3_15_R15000.db'
if not opacity_file.exists():
    import picaso.data as picaso_data
    picaso_data.get_data(category_download='resampled_opacity', target_download='default',
                         final_destination_dir=str(reference_dir / 'opacities'))
    if not opacity_file.exists():
        raise FileNotFoundError('The PICASO opacity database did not download.')




from picaso import justdoit as pdi
from virga import justdoit as vdi

#plotting tools
from picaso import justplotit as ppi
from virga import justplotit as vpi
ppi.output_notebook()
```

```
WARNING: Failed to load Vega spectrum from None; Functionality involving Vega will be severely
limited: FileNotFoundError(2, 'No such file or directory') [stsynphot.spectrum]
```

Interactive plot: open the solution notebook to explore this figure.

Now, since we don't have a model that dictates the particle size distribution of our expected haze layer, let's load in a sample particle size distribution. This one comes from He et al., 2018, on the sizes of hazes produced in a laboratory setting.

This is technically just relevant for the He et al water-rich haze optical properties, but we'll use them for both the He and Drant et al. nitrogen/methane/carbon monoxide derived haze here.

Notice that these are given in particle diameter, so we have to convert them to radii.

Load Custom Particle Radius Distribution

Code cell 6

```
from inspect import getsourcefile
import os
#if you haven't installed virga from github this file will exist in your libary package here
particle_distribution_file = os.path.join(

str(os.path.abspath(getsourcefile(vdi)).replace('justdoit.py','')),'reference','particle_sizes.csv')
#all this does is grab the code directory to get the reference data location of the code
```

Code cell 7

```
particle_distribution = pd.read_csv(particle_distribution_file)

n_400K, size =  particle_distribution['400 K'], particle_distribution['size (nm)']

#cut the range of sizes where there aren't any data points
non_nans_400 = np.where(np.isfinite(n_400K))


n4 = n_400K.iloc[non_nans_400]
r4 = size.iloc[non_nans_400]


#turn them into arrays for handy plotting and convert the particle diameters into radii
r4=np.array(r4/2)
n4=np.array(n4)
```

Let's plot up the distribution so we can see what it looks like.

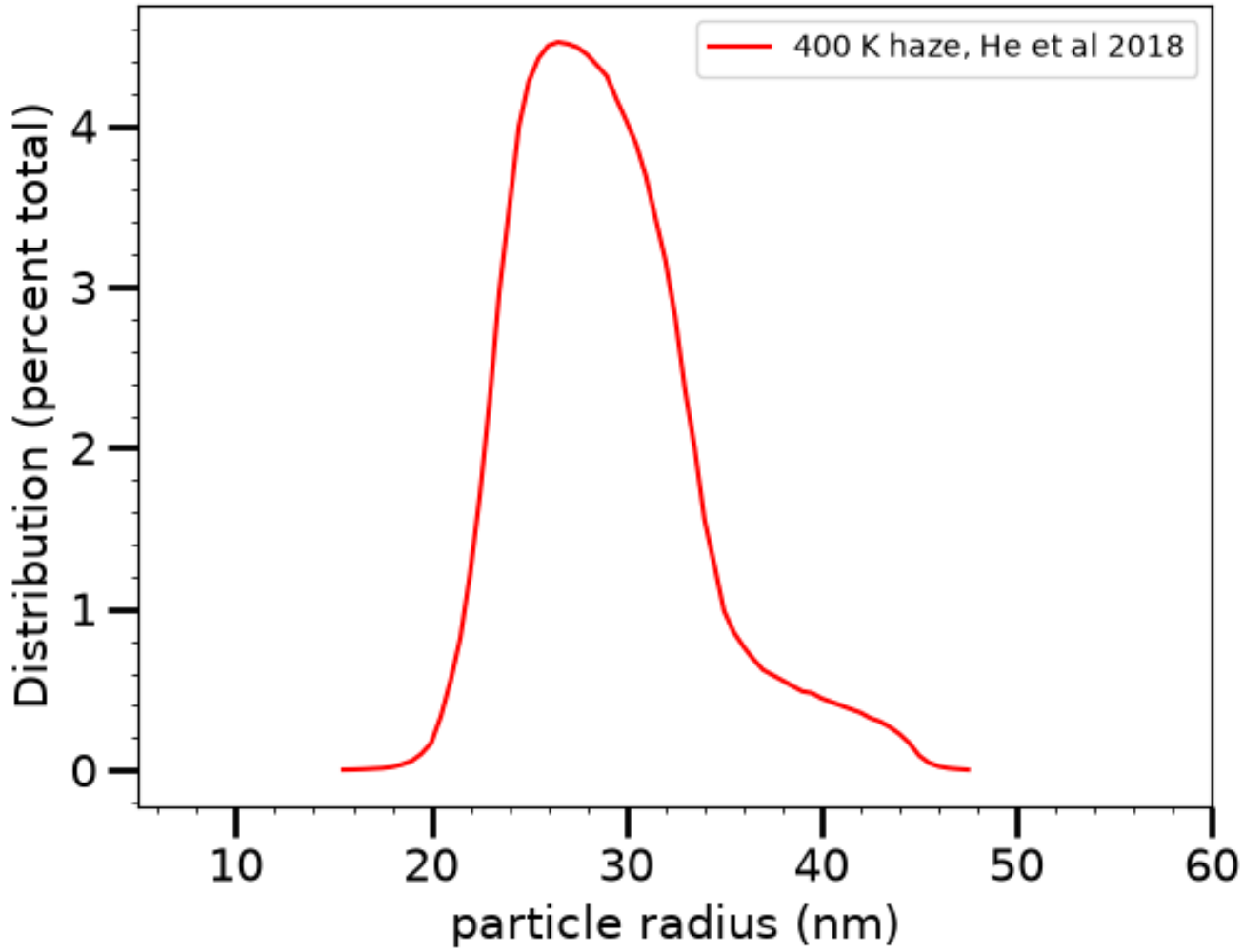
Code cell 9

```
plt.plot(r4,n4,color='red',label='400 K haze, He et al 2018')

plt.xlim(5,60)

plt.xlabel('particle radius (nm)',fontsize=15)
plt.ylabel('Distribution (percent total)',fontsize=15)
plt.minorticks_on()
plt.tick_params(axis='y',which='major',length =10, width=2,direction='out',labelsize=15)
plt.tick_params(axis='x',which='major',length =10, width=2,direction='out',labelsize=15)
plt.legend()
```

```
<matplotlib.legend.Legend at 0x12aa30dd0>
```



Okay, looks good! Now we're going to compute a new custom set of Mie coefficients that correspond to these particle radii. In future workflows, make sure you **stick these new Mie databases in a new folder so you don't overwrite your existing Virga databases!** Here, we'll stick them in a sub-folder of our JWebbinar folder.

Compute Mie Parameters for Custom Radius Grid

Code cell 12

```
#Remember to change these paths to where your refractive indices and Mie databases live
mieff_dir = str(workshop_dir / 'data' / 'processed' / 'custom_radii_grid')
# And point to where the nicely formatted .refrind file live from Notebook 1
refrind_dir = str(workshop_dir / 'data' / 'processed')

#First, let's just assume our hazes follow the distribution of the 400 K haze above.

#Note that this function wants the radii in centimeters, so we have to convert it from nanometers!
tholin = 'Drant_N2_CH4_CO_95_4.95_0.05_tholin'
newmies_drant=vdi.calc_mie_db(tholin, refrind_dir, mieff_dir,
                        rmin = np.min(r4/1e7), rmax=np.max(r4/1e7), nradii = len(r4))

tholin = 'He_400K_Water-rich_tholin'
newmies_he=vdi.calc_mie_db(tholin, refrind_dir, mieff_dir,
                        rmin = np.min(r4/1e7), rmax=np.max(r4/1e7), nradii = len(r4))
```

```
Computing optical properties for Drant_N2_CH4_CO_95_4.95_0.05_tholin using MiePython...
```

```
If this function seems to be running a really long time... Check that you gave your rmin (and r_mon if
 using aggregates) in centimeters and that you are not making tennis balls :)

360 wavelengths of refractive index data found for Drant_N2_CH4_CO_95_4.95_0.05_tholin. Grid of mean
radii (in bins) to calculate extinction and scattering efficiencies for (in cm):

           min             mean             max          bin width (dr)
       1.536439e-06    1.550000e-06    1.563561e-06       2.712164e-08
       1.563561e-06    1.577361e-06    1.591161e-06       2.760039e-08
       1.591161e-06    1.605205e-06    1.619249e-06       2.808760e-08
       1.619249e-06    1.633541e-06    1.647832e-06       2.858341e-08
       1.647832e-06    1.662376e-06    1.676920e-06       2.908798e-08
       1.676920e-06    1.691721e-06    1.706522e-06       2.960145e-08
       1.706522e-06    1.721584e-06    1.736646e-06       3.012398e-08
       1.736646e-06    1.751973e-06    1.767301e-06       3.065573e-08
       1.767301e-06    1.782900e-06    1.798498e-06       3.119688e-08
       1.798498e-06    1.814372e-06    1.830246e-06       3.174757e-08
       1.830246e-06    1.846400e-06    1.862554e-06       3.230799e-08
       1.862554e-06    1.878993e-06    1.895432e-06       3.287830e-08
       1.895432e-06    1.912161e-06    1.928891e-06       3.345868e-08
       1.928891e-06    1.945915e-06    1.962940e-06       3.404930e-08
       1.962940e-06    1.980265e-06    1.997590e-06       3.465034e-08
       1.997590e-06    2.015221e-06    2.032852e-06       3.526200e-08
       2.032852e-06    2.050795e-06    2.068737e-06       3.588446e-08
       2.068737e-06    2.086996e-06    2.105255e-06       3.651790e-08
       2.105255e-06    2.123836e-06    2.142417e-06       3.716252e-08
       2.142417e-06    2.161327e-06    2.180236e-06       3.781852e-08
       2.180236e-06    2.199479e-06    2.218722e-06       3.848611e-08
       2.218722e-06    2.238305e-06    2.257887e-06       3.916547e-08
       2.257887e-06    2.277816e-06    2.297744e-06       3.985683e-08
       2.297744e-06    2.318024e-06    2.338305e-06       4.056040e-08
       2.338305e-06    2.358943e-06    2.379581e-06       4.127638e-08
       2.379581e-06    2.400584e-06    2.421586e-06       4.200500e-08
       2.421586e-06    2.442959e-06    2.464333e-06       4.274649e-08
       2.464333e-06    2.486083e-06    2.507834e-06       4.350106e-08
       2.507834e-06    2.529968e-06    2.552103e-06       4.426895e-08
       2.552103e-06    2.574628e-06    2.597153e-06       4.505040e-08
       2.597153e-06    2.620076e-06    2.642999e-06       4.584564e-08
       2.642999e-06    2.666326e-06    2.689653e-06       4.665492e-08
       2.689653e-06    2.713393e-06    2.737132e-06       4.747848e-08
       2.737132e-06    2.761290e-06    2.785449e-06       4.831659e-08
       2.785449e-06    2.810033e-06    2.834618e-06       4.916948e-08
       2.834618e-06    2.859637e-06    2.884655e-06       5.003744e-08
       2.884655e-06    2.910116e-06    2.935576e-06       5.092071e-08
       2.935576e-06    2.961486e-06    2.987396e-06       5.181958e-08
       2.987396e-06    3.013763e-06    3.040130e-06       5.273431e-08
       3.040130e-06    3.066963e-06    3.093795e-06       5.366519e-08
       3.093795e-06    3.121102e-06    3.148408e-06       5.461250e-08
       3.148408e-06    3.176196e-06    3.203984e-06       5.557654e-08
       3.203984e-06    3.232263e-06    3.260542e-06       5.655759e-08
       3.260542e-06    3.289320e-06    3.318098e-06
[Long progress output shortened in PDF; full output is in the solution notebook.]
Optical properties for Drant_N2_CH4_CO_95_4.95_0.05_tholin have been calculated and saved as /Users/jj
m6243/Documents/jwebbinar_prep/jwebbinar53/data/processed/custom_radii_grid/Drant_N2_CH4_CO_95_4.95_0.
05_tholin.mieff.


Computing optical properties for He_400K_Water-rich_tholin using MiePython...
If this function seems to be running a really long time... Check that you gave your rmin (and r_mon if
 using aggregates) in centimeters and that you are not making tennis balls :)

512 wavelengths of refractive index data found for He_400K_Water-rich_tholin. Grid of mean radii (in
bins) to calculate extinction and scattering efficiencies for (in cm):

           min             mean             max          bin width (dr)
       1.536439e-06    1.550000e-06    1.563561e-06       2.712164e-08
       1.563561e-06    1.577361e-06    1.591161e-06       2.760039e-08
```

```
1.591161e-06    1.605205e-06    1.619249e-06       2.808760e-08
1.619249e-06    1.633541e-06    1.647832e-06       2.858341e-08
1.647832e-06    1.662376e-06    1.676920e-06       2.908798e-08
1.676920e-06    1.691721e-06    1.706522e-06       2.960145e-08
1.706522e-06    1.721584e-06    1.736646e-06       3.012398e-08
1.736646e-06    1.751973e-06    1.767301e-06       3.065573e-08
1.767301e-06    1.782900e-06    1.798498e-06       3.119688e-08
1.798498e-06    1.814372e-06    1.830246e-06       3.174757e-08
1.830246e-06    1.846400e-06    1.862554e-06       3.230799e-08
1.862554e-06    1.878993e-06    1.895432e-06       3.287830e-08
1.895432e-06    1.912161e-06    1.928891e-06       3.345868e-08
1.928891e-06    1.945915e-06    1.962940e-06       3.404930e-08
1.962940e-06    1.980265e-06    1.997590e-06       3.465034e-08
1.997590e-06    2.015221e-06    2.032852e-06       3.526200e-08
2.032852e-06    2.050795e-06    2.068737e-06       3.588446e-08
2.068737e-06    2.086996e-06    2.105255e-06       3.651790e-08
2.105255e-06    2.123836e-06    2.142417e-06       3.716252e-08
2.142417e-06    2.161327e-06    2.180236e-06       3.781852e-08
2.180236e-06    2.199479e-06    2.218722e-06       3.848611e-08
2.218722e-06    2.238305e-06    2.257887e-06       3.916547e-08
2.257887e-06    2.277816e-06    2.297744e-06       3.985683e-08
2.297744e-06    2.318024e-06    2.338305e-06       4.056040e-08
2.338305e-06    2.358943e-06    2.379581e-06       4.127638e-08
2.379581e-06    2.400584e-06    2.421586e-06       4.200500e-08
2.421586e-06    2.442959e-06    2.464333e-06       4.274649e-08
2.464333e-06    2.486083e-06    2.507834e-06       4.350106e-08
2.507834e-06    2.529968e-06    2.552103e-06       4.426895e-08
2.552103e-06    2.574628e-06    2.597153e-06       4.505040e-08
2.597153e-06    2.620076e-06    2.642999e-06       4.584564e-08
2.642999e-06    2.666326e-06    2.689653e-06       4.665492e-08
2.689653e-06    2.713393e-06    2.737132e-06       4.747848e-08
2.737132e-06    2.761290e-06    2.785449e-06       4.831659e-08
2.785449e-06    2.810033e-06    2.834618e-06       4.916948e-08
2.834618e-06    2.859637e-06    2.884655e-06       5.003744e-08
2.884655e-06    2.910116e-06    2.935576e-06       5.092071e-08
2.935576e-06    2.961486e-06    2.987396e-06       5.181958e-08
2.987396e-06    3.013763e-06    3.040130e-06       5.273431e-08
3.040130e-06    3.066963e-06    3.093795e-06       5.366519e-08
3.093795e-06    3.121102e-06    3.148408e-06
[Long progress output shortened in PDF; full output is in the solution notebook.]
Optical properties for He_400K_Water-rich_tholin have been calculated and saved as /Users/jjm6243/Docu
ments/jwebbinar_prep/jwebbinar53/data/processed/custom_radii_grid/He_400K_Water-rich_tholin.mieff.
```

Okay, now let's grab those Mieff coefficients for the Drant data first.

Code cell 14

```
tholin = 'Drant_N2_CH4_CO_95_4.95_0.05_tholin'
qext, qscat, cos_qscat, nwave, radius, wave_in = vdi.get_mie(tholin,directory=mieff_dir)
```

Next, we calculate our optical properties in terms of the optical depth, single scattering albedo, and asymmetry. Remember that **r4** here is your grid of radii bins and **n4** is the distribution of particles per radius bin.

ndz is the total number of particles that make up our aerosol layer. Since we're setting our own slab, it's not really physical in this implementation because we're setting the particle number over many atmospheric layers. So we're going to set it fairly high. But ndz is essentially just a free tunable parameter here.

Code cell 16

```
ndz = 2e11 #that's particles/cm^2 but over the entire region of atmosphere where we're sticking
aerosol

opd,w0,g0,wavenumber_grid=vdi.calc_optics_user_r_dist(wave_in, ndz ,r4, u.nm, n4/100,
                                                      qext, qscat, cos_qscat, )
```

Now, we need to turn the above variables into something that PICASO can read. Choose where to set your aerosol layer pressure limits.

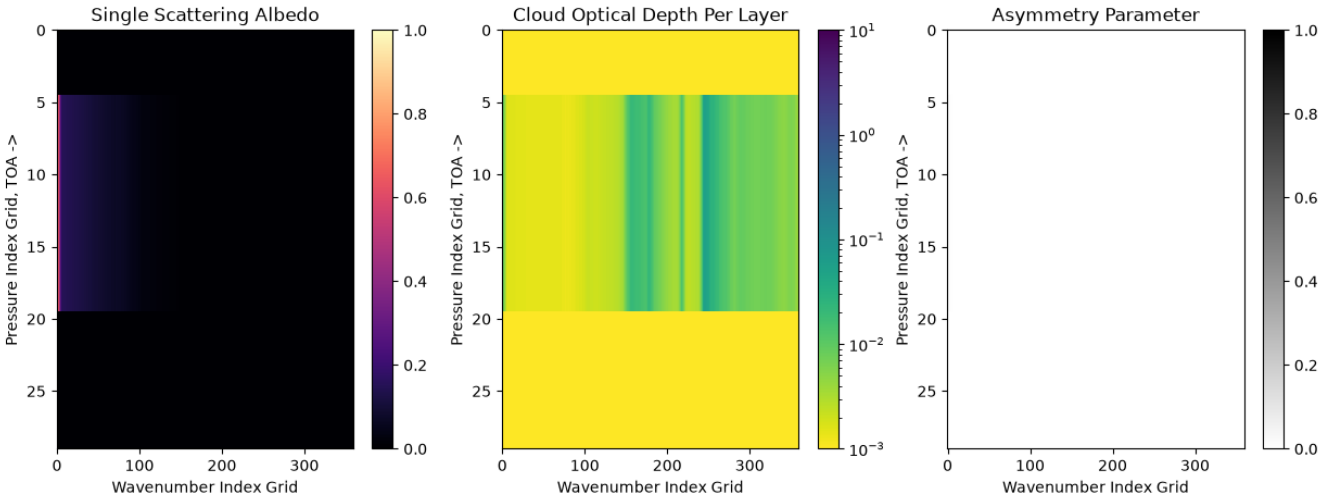
Code cell 18

```
nlayer = 30
base_pressure = 1e-1 # let's start by setting the haze layer base at 0.1 bars
haze_top_pressure = 1e-7 #and set the top of the haze layer at 0.1 microbar. Not unreasonble for a
haze that forms fairly high!

#just set an arbitrary pressure grid from 1000 bars to a nanobar.
# This is a simpler model than we explored in the brown dwarf!
pressure =  np.logspace(-9,3,nlayer)

#here's where we shove all the variables into a dataframe PICASO can read
df_haze_drant = vdi.picaso_format_slab(base_pressure,opd, w0, g0, wavenumber_grid,
pressure,p_top=haze_top_pressure)

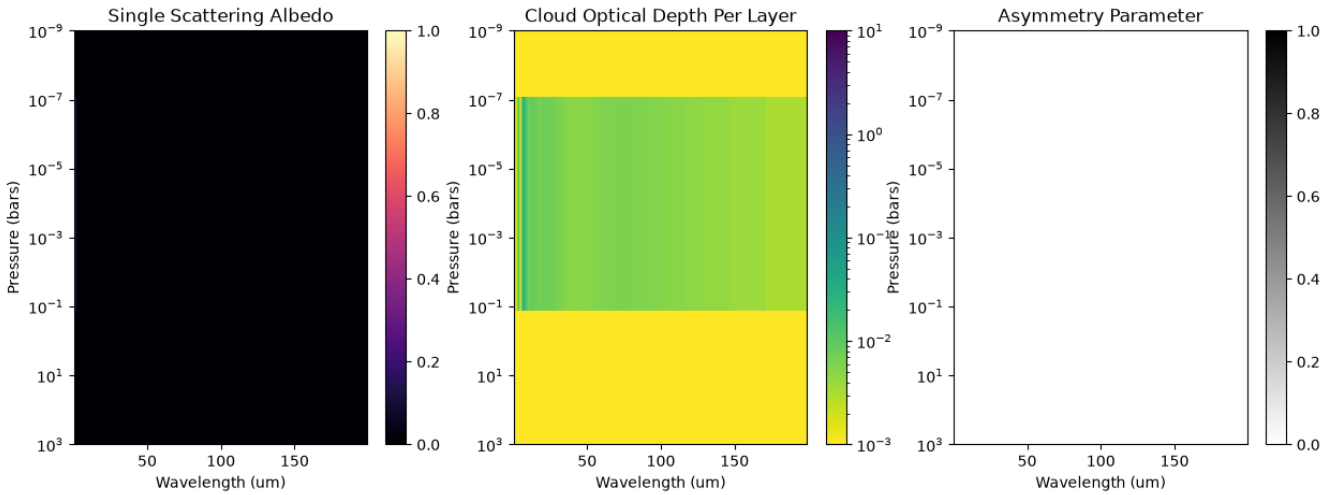

nwno = len(wavenumber_grid) #this will depend on the refractive indices you have. Khare's tholin have
wide coverage but low resolution
ppi.plot_cld_input(nwno, nlayer, df=df_haze_drant)
plt.show()
```



We can see that slab of haze and how it's changing according to wavelength! The above plots aren't in physical units though. Let's change the axes to be real numbers and not gridpoints. The wavelength grid covers a lot, so you'll probably have to zoom in to see the wavelength units.

Code cell 20

```
ppi.plot_cld_input(nwno, nlayer, df=df_haze_drant, pressure=pressure, wavelength=1e4/wavenumber_grid)
plt.show()
```



Let's compare these Drant data to the same particle size distribution, haze mass loading, and haze pressure extent but now using the He et al data for a water-rich derived haze.

Code cell 22

```
tholin = 'He_400K_Water-rich_tholin'
qext, qscat, cos_qscat, nwave, radius, wave_in = vdi.get_mie(tholin,directory=mieff_dir)

ndz = 2e11 #that's particles/cm^2 but over the entire region of atmosphere where we're sticking
aerosol

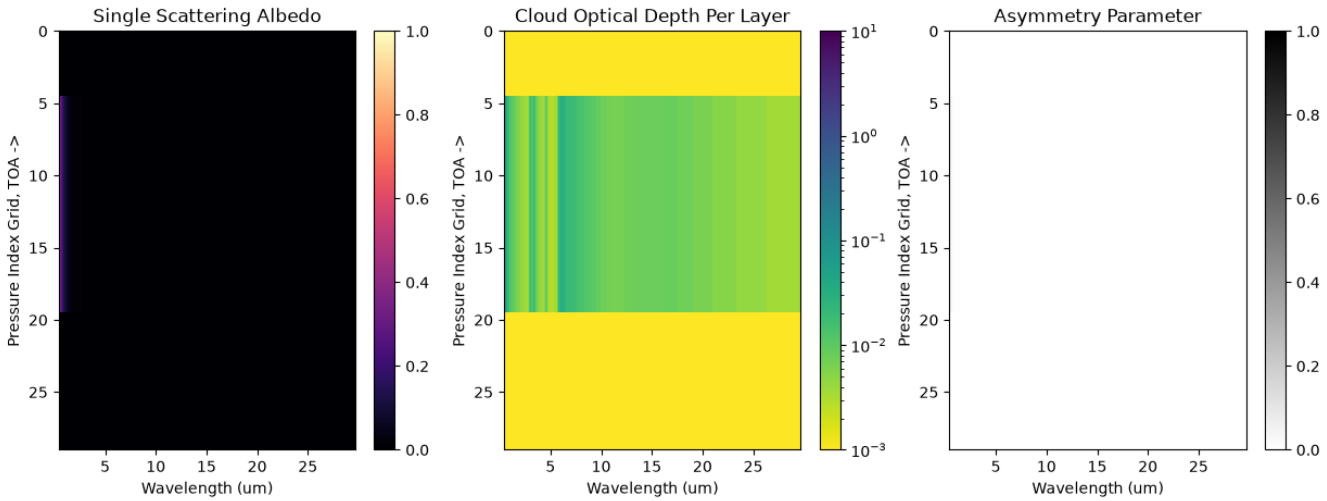
opd,w0,g0,wavenumber_grid=vdi.calc_optics_user_r_dist(wave_in, ndz ,r4, u.nm, n4/100,
                                                      qext, qscat, cos_qscat, )

nlayer = 30
base_pressure = 1e-1 # let's start by setting the haze layer base at 0.1 bars
haze_top_pressure = 1e-7 #and set the top of the haze layer at 0.1 microbar. Not unreasonble for a
haze that forms fairly high!

#just set an arbitrary pressure grid from 1000 bars to a nanobar.
# This is a simpler model than we explored in the brown dwarf!
pressure =  np.logspace(-9,3,nlayer)

#here's where we shove all the variables into a dataframe PICASO can read
df_haze_he = vdi.picaso_format_slab(base_pressure,opd, w0, g0, wavenumber_grid,
pressure,p_top=haze_top_pressure)


nwno = len(wavenumber_grid) #this will depend on the refractive indices you have. Khare's tholin have
wide coverage but low resolution
ppi.plot_cld_input(nwno, nlayer, df=df_haze_he, wavelength=1e4/wavenumber_grid)
plt.show()
```



Already we can see that, despite the optical properties and Mie efficiencies of these two datasets looking similar from Notebook 1, they propagate to very different haze profiles even if we keep every other aspect of the haze the same!

Now let's see how these would look in a spectrum and how it compares to real data. We're going to look at GJ 1214 b, the most famously hazy sub-Neptune. We'll look specifically at the JWST NIRSpec G395H spectrum from Schlawin et al.,2024. First, let's load a couple basic planet parameters.

Code cell 24

```
file = str(workshop_dir / 'data' / 'observations' / 'table_GJ-1214-b-Schlawin-et-al.-2024.tsv')
data = pd.read_csv(file,sep='\t')


nlevel = nlayer+1
opa = pdi.opannection(wave_range=[0.3,14],filename_db=str(reference_dir / 'opacities' /
'opacities_0.3_15_R15000.db'))
case1 = pdi.inputs()

case1.phase_angle(0)


#here we are going to have to specify gravity through R and M since we need it in the Flux calc
# We're using the parameters specific to GJ 1214 b
case1.gravity(mass=0.0265, mass_unit=pdi.u.Unit('M_jup'),
              radius=0.2438, radius_unit=pdi.u.Unit('R_jup'))

#here we are going to have to specify R as well
case1.star(opa, 3500,0.2,5.0,radius=0.22, radius_unit = pdi.u.Unit('R_sun'))

#atmo -- make sure your pressure grid matches the one you computed your haze on!
# our chemistry here is entirely arbitrary to make sure that
# we get molecular features -- we're not interpreting the spectrum for real!

case1.atmosphere( df = pdi.pd.DataFrame({'pressure':np.logspace(-9,3,nlevel),
                                         'temperature':np.logspace(-9,3,nlevel)*0+600, #just
an isothermal one for simplicity
                                         "H2O":np.logspace(-9,3,nlevel)*0+0.66,
                                         "He":np.logspace(-9,3,nlevel)*0+0.16,
                                         "CO2":np.logspace(-9,3,nlevel)*0+0.049,
                                         "N2":np.logspace(-9,3,nlevel)*0+0.065,
```

```
                                                "CH4":np.logspace(-9,3,nlevel)*0+0.066}))
```

And compute and plot our clear transmission spectrum, without our haze:

Code cell 26

```
df= case1.spectrum(opa, full_output=True,calculation='transmission')

wno, rprs2  = df['wavenumber'] , df['transit_depth']
wno_clear, rprs2_clear = pdi.mean_regrid(wno, rprs2, R=300)
full_output = df['full_output']



plt.figure(figsize=(15, 9))
plt.rcParams['axes.labelsize'] = 22 # X and Y label size
plt.rcParams['xtick.labelsize'] = 20 # X tick label size
plt.rcParams['ytick.labelsize'] = 20 # Y tick label size

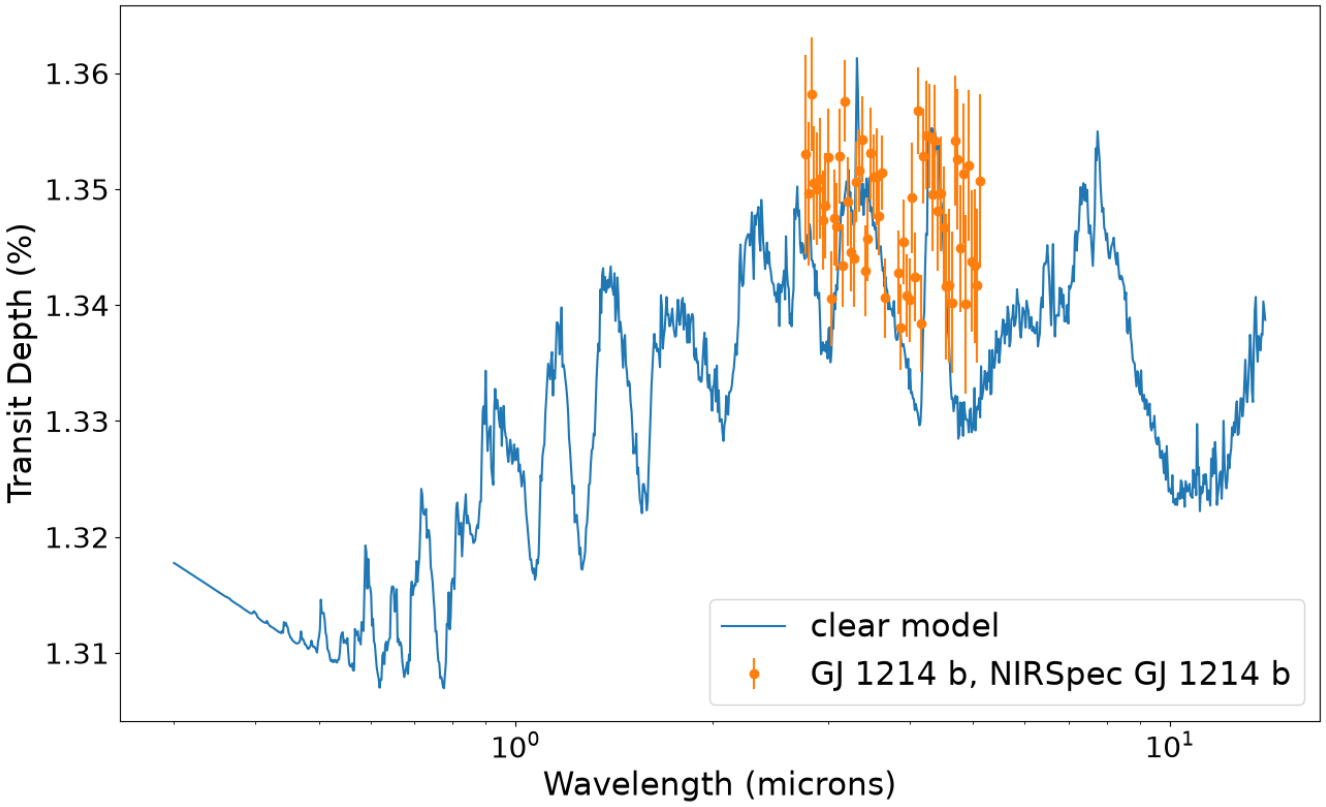

#picaso naturally computes in wavenumbers, so convert to microns and get in transit depth as a percent
plt.plot(1e4/wno_clear,rprs2_clear*100,label="clear model")
plt.errorbar(np.array(data['CENTRALWAVELNG']),np.array(data['PL_TRANDEP']),
             yerr=np.array(data['PL_TRANDEPERR1']),fmt='o',label="GJ 1214 b, NIRSpec GJ 1214 b")

plt.legend(loc=4,fontsize=22,ncol=1)
plt.xscale('log')
plt.ylabel('Transit Depth (%)')
plt.xlabel('Wavelength (microns)')



```

```
Text(0.5, 0, 'Wavelength (microns)')
```



We can see that our spectrum is far too large to fit the actual data. Now, let's add in that hazy information.

Code cell 28

```
case1.clouds(df=df_haze_drant.astype(float))
hazy= case1.spectrum(opa, full_output=True,calculation='transmission')
hazyx,hazyy =hazy['wavenumber'] , hazy['transit_depth']
hazyx_drant,hazyy_drant = pdi.mean_regrid(hazyx,hazyy, R=300)


case1.clouds(df=df_haze_he.astype(float))
hazy= case1.spectrum(opa, full_output=True,calculation='transmission')
hazyx,hazyy =hazy['wavenumber'] , hazy['transit_depth']
hazyx_he,hazyy_he = pdi.mean_regrid(hazyx,hazyy, R=300)
```

Let's compare our hazy spectrum to our clear spectrum!

Code cell 30

```
df= case1.spectrum(opa, full_output=True,calculation='transmission')

wno, rprs2  = df['wavenumber'] , df['transit_depth']
wno, rprs2 = pdi.mean_regrid(wno, rprs2, R=300)
full_output = df['full_output']


plt.figure(figsize=(15, 9))
plt.rcParams['axes.labelsize'] = 22 # X and Y label size
plt.rcParams['xtick.labelsize'] = 20 # X tick label size
plt.rcParams['ytick.labelsize'] = 20 # Y tick label size

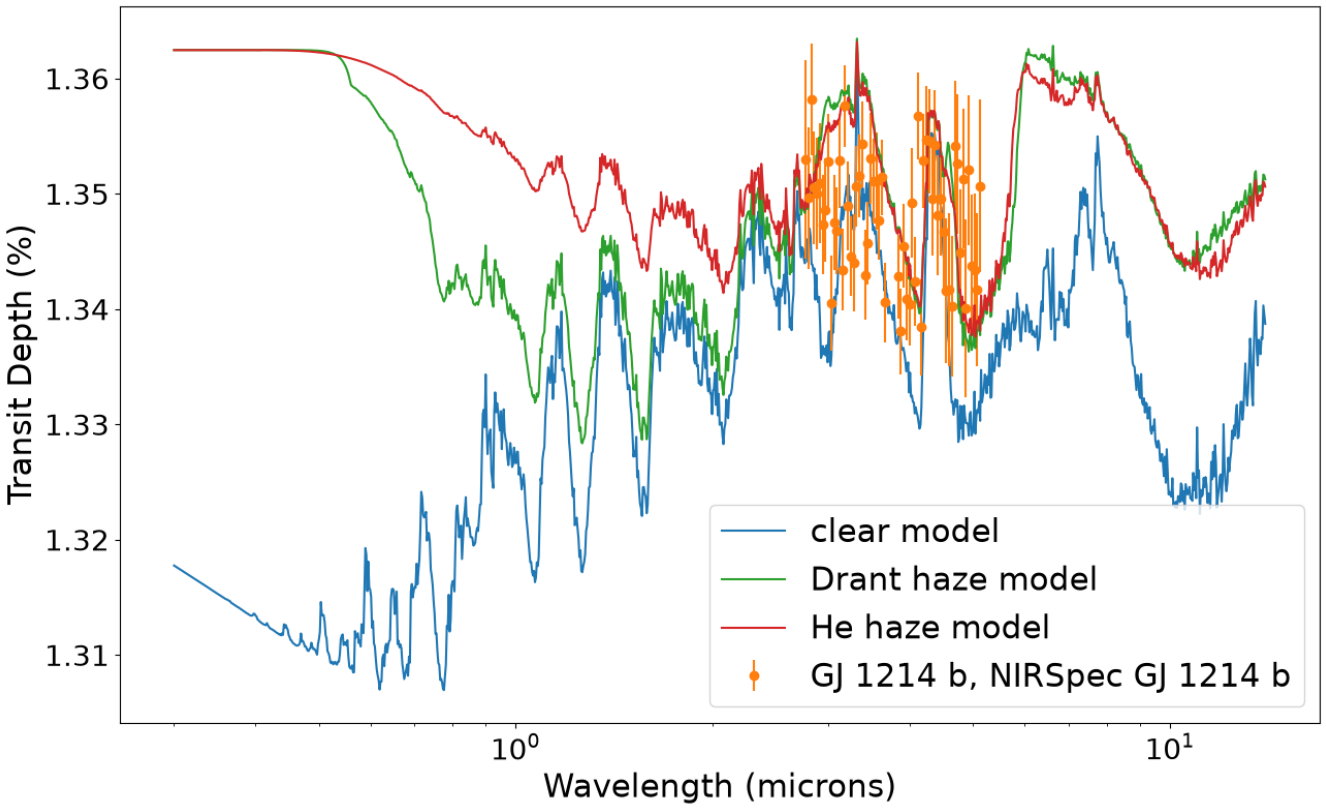

#picaso naturally computes in wavenumbers, so convert to microns and get in transit depth as a percent
plt.plot(1e4/wno_clear,rprs2_clear*100,label="clear model")
plt.errorbar(np.array(data['CENTRALWAVELNG']),np.array(data['PL_TRANDEP']),
             yerr=np.array(data['PL_TRANDEPERR1']),fmt='o',label="GJ 1214 b, NIRSpec GJ 1214 b")

plt.plot(1e4/hazyx_drant,hazyy_drant*100,label="Drant haze model")
plt.plot(1e4/hazyx_he,hazyy_he*100,label="He haze model")


plt.legend(loc=4,fontsize=22,ncol=1)
plt.xscale('log')
plt.ylabel('Transit Depth (%)')
plt.xlabel('Wavelength (microns)')


```

```
Text(0.5, 0, 'Wavelength (microns)')
```



In the visible, the haze creates a bigger scattering slope and mutes the water, methane, and carbon dioxide features at NIR wavelengths. This is true for both sets of optical properties, but you can tell the difference between them! In the NIRSpec range, both sets of optical properties do a pretty good job of fitting the data within errorbars, but at NIRISS wavelengths, they diverge a lot.

With this amount of haze, we can also actually see IR features in the spectrum that are due to the haze itself. From 2.8 to almost 4 microns we see a large, broad amine (N-H) feature that overlays the gaseous methane feature, and at 5.5-6 microns we see a strong C-O or C-N stretch.

Things to try

- Change the "ndz" parameter to see if you can totally flatten the spectrum with the haze mass loading. Can you find the abundance of haze where it no longer affects the spectrum?
- Try changing the particle size distribution and seeing how that impacts your optical depth and spectra (try a linear multiplicative factor to the radii given, widen the distribution, make up a whole new one, etc.)
- Compare the JWST observation and the actual statistical fits of the models, e.g., with a chi-squared metric.
- Can you find other data for GJ 1214b and do a similar comparison?
- Go find another set of gas tholins on OCDb and repeat this process!

The Virga cloud and PICASO spectrum steps follow the linked tutorials, as with the Brown Dwarf Water Clouds notebook. Cite these models and the sources of the JWST data in scientific work, along with your chosen optical properties!